

# **BHV\_FixedTurn and BHV\_MemoryTurnLimit**

Turn-related behaviors.

## **Included MIT MOOS-IvP PDF sources:**

- bhv\_fixedturn.pdf
- bhv\_mem\_turnlimit.pdf

# The FixedTurn Behavior

Fall 2024

Michael Benjamin, mikerb@mit.edu  
Department of Mechanical Engineering  
MIT, Cambridge MA 02139  
project-pavlab/bhvdocs/bhv\_fixedturn

---

|          |                                                 |          |
|----------|-------------------------------------------------|----------|
| <b>1</b> | <b>The FixedTurn Behavior</b>                   | <b>1</b> |
| 1.1      | Intended Mission Use . . . . .                  | 2        |
| 1.2      | The Core Algorithm . . . . .                    | 2        |
| 1.3      | Configuration and Operation Mechanics . . . . . | 4        |
| 1.4      | Configuration Parameters . . . . .              | 5        |
| 1.5      | Variables Published . . . . .                   | 6        |
| 1.6      | Flags and Macros . . . . .                      | 6        |

---

## Contents

### 1 The FixedTurn Behavior

The *FixedTurn* behavior allows a vehicle to execute a turn with parameters for configuring the duration of the turn (in degrees), and the commanded changes in desired heading during the turn. The behavior is primarily motivated by supporting surface vehicle shake-out tests, but may be used in any mission where this pattern is desired. The behavior can be configured with varying number of turns, order of turns, and the number of degrees for each turn, and the manner in which the turn is requested.

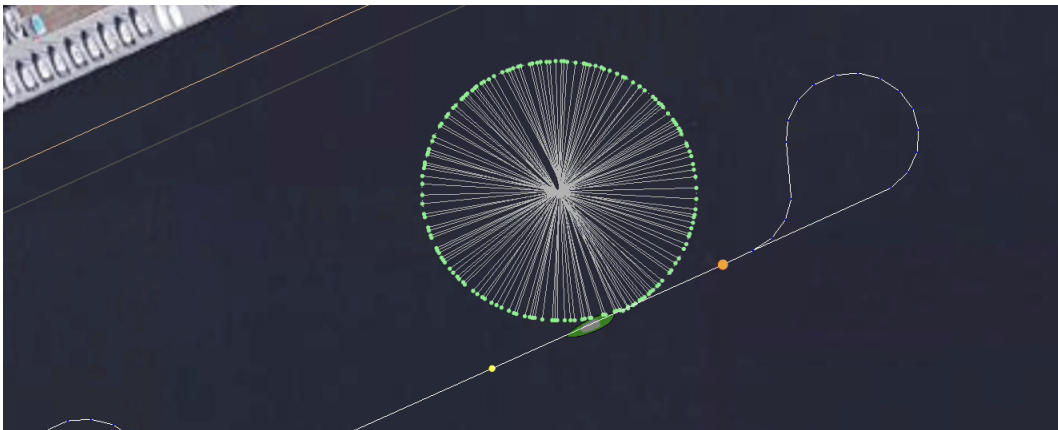


Figure 1: **FixedTurn Behavior:** A vehicle executes a FixedTurn behavior, with a single turn shown. The behavior starts and finishes within a pause of the LegRun behavior using the basic LegRun ability to initiate an event at given point in a leg. (From demo mission `missions-auto/04-fixedturn`).

Turns in a FixedTurn behavior are accomplished by continuously requesting a heading change of a configured fixed value, monitoring the number of degrees in a turn, until the turn completion criteria is reached. Completion criteria is in number of degrees, e.g, a 180 degree turn, or a 270 degree turn.

## 1.1 Intended Mission Use

A FixedTurn behavior is configured with a sequence of parameterized turns. A turn configuration will prescribe the turn speed, the fixed delta heading, the total number of degrees for the turn, and the turn direction. For example, the following two turns only differ in direction.

```
turn_spec = spd=1.5, mhdg=10, fix=355, turn=star
turn_spec = spd=1.5, mhdg=10, fix=355, turn=port
```

Once the turn has been completed, an end flag is published, presumably resulting in the FixedTurn behavior becoming idle. The next time the FixedTurn behavior enters the running state, the next turn in the list of turns will be executed. Presumably, in the meanwhile, while in the idle state, other behaviors in the mission will maneuver the vehicle to a position where the nexted turn is safe for execution. In the intended use case for this behavior, there is typically no obstacle or collision avoidance behaviors running during the turn. The mission author of course is free to do otherwise. However, the behavior was conceived to support the goal of conducting a controlled turn in a safe, hazard-free space, to gauge vehicle performance.

## 1.2 The Core Algorithm

The basic FixedTurn behavior will be configured with a sequence of turns, where each turn can be configured in terms of turn direction and size. The intended use for this behavior is for it to be run as a mission interruption of sorts, while the behavior is otherwise transiting. In the example in Figure 3, the turn is executing in the middle of the long straight leg of the vehicle running the LegRun behavior. This is from Example Mission N.

### OnRunState() Flow Chart

The primary operation of the behavior is contained within the OnRunState() function. This is shown the flow chart and the pseudocode that follows. **Step 1** of this sequence is to update the current value of ownship position and heading, a common first step in many behaviors. For this behavior, ownship heading is the key value for reasoning about turn completion.

### OnRunState() Pseudo Code

---

#### Algorithm 1: FixedTurn Behavior Main Loop

---

```
1: procedure ONRUNSTATE()
2:    $V \leftarrow \text{updatePositions}()$ 
```

▷ Step 1

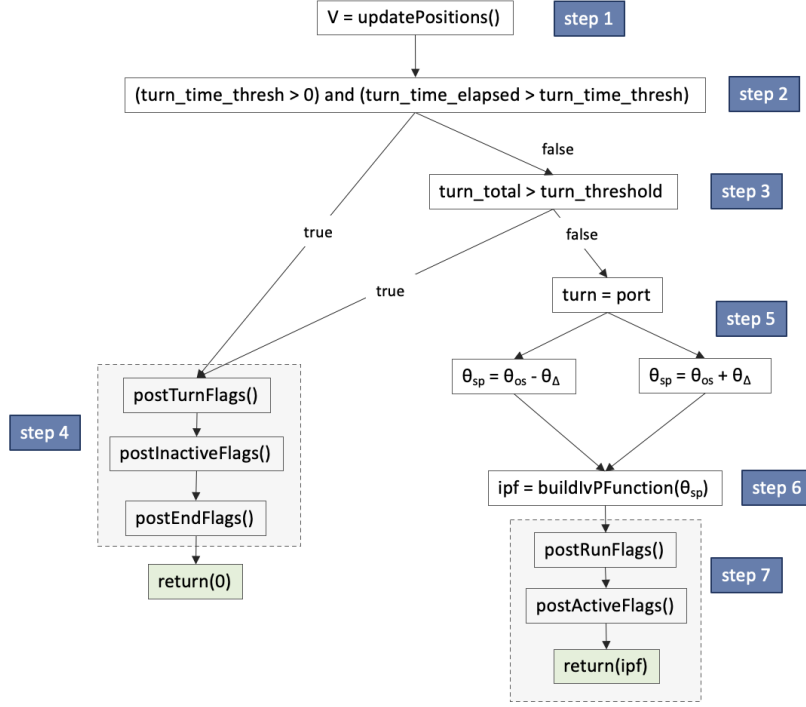


Figure 2: **FixedTurn Behavior Loop Flow Chart:** Each iteration of the FixedTurn behavior will proceed through the steps shown, with branching determined by sensed events and the behavior configuration. Details of each step are discuss in their own sections as indicated.

|                                                            |             |
|------------------------------------------------------------|-------------|
| 3: <b>if</b> timeout or turn_complete then                 | ▷ Steps 2,3 |
| 4:       postTurnFlags()                                   | ▷ Step 4    |
| 5:       postRunFlags()                                    | ▷ Step 4    |
| 6:       postEndFlags()                                    | ▷ Step 4    |
| 7: <b>return</b> 0                                         |             |
| 8: <b>end if</b>                                           |             |
| 9: <b>if</b> turn = port then                              | ▷ Step 5    |
| 10: $\theta_{sp} \leftarrow \theta_{sp} - \theta_{\Delta}$ |             |
| 11: <b>else</b>                                            |             |
| 12: $\theta_{sp} \leftarrow \theta_{sp} + \theta_{\Delta}$ |             |
| 13: <b>end if</b>                                          |             |
| 14: $ipf \leftarrow \text{buildIvPFunction}(\theta_{sp})$  | ▷ Step 6    |
| 15:   postRunFlags()                                       | ▷ Step 7    |
| 16:   postActiveFlags()                                    | ▷ Step 7    |
| 17: <b>return</b> ipf                                      |             |
| 18: <b>end procedure</b>                                   |             |

In **Step 2**, the behavior checks to see if there is a timeout threshold specified for this turn. If there is, the timer is started as soon as the behavior transitions into the running state. The elapsed time since then is compared to the timeout threshold and, if it is exceeded, the behavior regards the turn as completed. In **Step 3**, the turn is checked for completion based on the turn degrees specified, and how much the vehicle has turned thus far. Although a full 360 degree turn is common, the

specified turn may be any value, even beyond 360 degrees.

In **Step 4**, a behavior completion sequence is conducted. This includes the posting of end flags and inactive flags, if they are provided by the user in the behavior configuration. As well as turn flags, which are separate flags supported uniquely for this behavior, posted whenever a turn has been completed.

In **Step 5**, depending on whether the current turn is to the port or starboard, the desired heading set point  $\theta_{sp}$ , is set to be offset from ownship current heading  $\theta_{OS}$  by the number of degrees,  $\theta_{\Delta}$  prescribed in the configuration parameter `mod_hdg`. This heading set point is passed, in **Step 6** to the function where the IvP objective function (ipf) is created. See Section ??.

In **Step 7**, finally the run flags and active flags are posted this at this stage the behavior is in both the running (non-idle) and active (generates an IvP Function) state. The IvP function is returned to the helm to contribute to the selection of the heading and speed for the current helm iteration.

## 1.3 Configuration and Operation Mechanics

### 1.3.1 Configuring the Turns

The `FixedTurn` behavior is configured with one or more *turns*, constituting a *turn sequence*. A single turn is comprised of the following *turn elements*:

- *turn length*: number of degrees constituting a turn
- *heading offset*: requested change of heading during the turn
- *turn speed*: desired speed during the turn
- *turn direction*: port or starboard
- *turn timeout*: a max time, in seconds, allowed for the turn
- *turn flags*: one or more flags to be posted at turn completion.

```
turn_spec = spd=2.5, mhdg=20, fix=360, turn=star, timeout=60
turn_spec = spd=2.5, mhdg=20, fix=360, turn=port, timeout=60
turn_spec = spd=4.0, mhdg=10, fix=180, turn=star, flag=STAGE=complete
turn_spec = spd=4.0, mhdg=10, fix=180, turn=port, flag=STAGE=complete
```



Figure 3: **Back-to-back Turns:** A vehicle executes a FixedTurn mission `s6_1egrun`).

## 1.4 Configuration Parameters

*Listing 1.1: Configuration Parameters for the FixedTurn Behavior.*

| Parameter                      | Description                                                                                                                                                             |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>fix_turn:</code>         | The default turn threshold for any turn that does not provide one. The default value is 0 degrees. Section 1.3.1.                                                       |
| <code>mod_hdg:</code>          | The default heading offset, $\theta_{\Delta}$ for any turn that does not specify a heading offset. The default value is 20 degrees. Section 1.3.1.                      |
| <code>turn_dir:</code>         | The default turn direction for any turn that does specify a direction. The default direction is <i>port</i> . Section 1.3.1.                                            |
| <code>speed:</code>            | The default speed for any turn that does not specify a turn speed. The default value is 1 meter per second. Section 1.3.1.                                              |
| <code>timeout:</code>          | The default timeout duration, in seconds, for any turn that does not specify one. The default value is -1, indicating there is not timeout for the turn. Section 1.3.1. |
| <code>turn_delay:</code>       | Duration in seconds at the start of turn before requesting a heading change, presumably to allow for a speed adjustment before turning.                                 |
| <code>turn_spec:</code>        | A full specification for a scheduled turn. Section 1.3.1.                                                                                                               |
| <code>stale_nav_thresh:</code> | The number of seconds, after which an update nav position and heading has not been received, before the behavior throws a warning. Section 1.3.1.                       |
| <code>schedule_repeat:</code>  | The number of times the full list of turns in the turn schedule will be repeated. The default is zero. Section 1.3.1.                                                   |
| <code>visual_hints:</code>     | Color and other settings for posted visual artifacts. Section 1.3.1.                                                                                                    |

*Listing 1.2: Example Configuration Block.*

```

Behavior = BHV_FixedTurn
{
    name      = zig
    pwt       = 100
    condition = MODE==FTURNING
    endflag   = FIX_TURN = false

    updates   = FTURN_UPDATE
    perpetual = true

    fix_turn   = tbd
    mod_hdg    = 20
    turn_dir   = star
    speed      = 2
    timeout    = 60
    turn_delay = 10

    turn_spec = spd=auto, mhdg=10, fix=355, turn=auto, flag=FIX_TURN=false
    turn_spec = spd=auto, mhdg=30, fix=355, turn=port, flag=FIX_TURN=false

    stale_nav_thresh = 5
    schedule_repeat   = true
    visual_hints      = vertex_color=dodger_blue
    visual_hints      = vertex_size=4
}

```

## 1.5 Variables Published

The below MOOS variables will be published by the behavior during normal operation, in addition to any configured flags.

- **VIEW\_SEGLIST**: A visual cue for rendering the requested zig angle, if either `draw_set_hdg=true` or `draw_req_hdg=true` in the `visual_hints`.
- **FT\_REPORT**: A report posted at the end of each turn, with parameter for the turn and observed turn radius information. An example is given below.

```
FT_REPORT = spd=0.88,mod_hdg=10,rudder_avg=10.7,avg_rad=55.5,min_rad=54.23,max_rad=56.1}
```

## 1.6 Flags and Macros

The FixedTurn behavior supports all the normal behavior flags, and a custom flag(s) can be added to the end of each turn.

The following macros are supported in the FixedTurn behavior. These macros will be expanded in any event flag, including event flags defined for all IvP behaviors as well as event flags defined only for the FixedTurn behavior.

- `$(TURN_DIST)`: Total odometry in the current or most recent turn.

- $\$[TURN\_TIME]$ : Total time since the beginning of the current or most recent turn.



# The MemoryTurnLimit Behavior

Fall 2024

Michael Benjamin, mikerb@mit.edu  
Department of Mechanical Engineering  
MIT, Cambridge MA 02139  
project-pavlab/bhvdocs/bhv\_mem\_turnlimit

---

|          |                                                  |          |
|----------|--------------------------------------------------|----------|
| <b>1</b> | <b>The MemoryTurnLimit Behavior</b>              | <b>1</b> |
| 1.1      | Configuration Parameters . . . . .               | 1        |
| 1.2      | Calculation of the Heading History . . . . .     | 3        |
| 1.3      | Variables Published . . . . .                    | 3        |
| 1.4      | The MemoryTurnLimit Objective Function . . . . . | 3        |

---

## Contents

## 1 The MemoryTurnLimit Behavior

The objective of the Memory-Turn-Limit behavior is to avoid vehicle turns that may cross back on its own path and risk damage to the towed equipment. Its configuration is determined by the two parameters described below which combine to set a vehicle turn radius limit. However, it is not strictly described by a limited turn radius; it stores a time-stamped history of recent recorded headings and maintains a *heading average*, and forms its objective function on a range deviation from that average. This behavior merely expresses a preference for a particular heading. If other behaviors also have a heading preference, coordination/compromise will take place through the multi-objective optimization process. The following parameters are defined for this behavior:

### 1.1 Configuration Parameters

*Listing 1.1: Configuration Parameters Common to All Behaviors.*

|                             |                                                                                                                   |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>activeflag:</b>          | A MOOS variable-value pair posted when the behavior is in the <i>active</i> state. <a href="#">[more]</a> .       |
| <b>condition:</b>           | Specifies a condition that must be met for the behavior to be running. <a href="#">[more]</a> .                   |
| <b>duration:</b>            | Time in behavior will remain running before declaring completion. <a href="#">[more]</a> .                        |
| <b>duration_idle_decay:</b> | When true, duration clock is running even when in the <i>idle</i> state. <a href="#">[more]</a> .                 |
| <b>duration_reset:</b>      | A variable-pair such as <code>MY_RESET=true</code> , that will trigger a duration reset. <a href="#">[more]</a> . |
| <b>duration_status:</b>     | The name of a MOOS variable to which the vehicle duration status is published. <a href="#">[more]</a> .           |

|                             |                                                                                                                   |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>endflag</code> :      | A MOOS variable-value pair posted when the behavior has completed. <a href="#">[more]</a> .                       |
| <code>idleflag</code> :     | A MOOS variable-value pair posted when the behavior is in the <i>idle</i> state. <a href="#">[more]</a> .         |
| <code>inactiveflag</code> : | A MOOS variable-value posted when the behavior is <i>not</i> in the <i>active</i> state. <a href="#">[more]</a> . |
| <code>name</code> :         | The (unique) name of the behavior. <a href="#">[more]</a> .                                                       |
| <code>nostarve</code> :     | Allows a behavior to assert a maximum staleness for a MOOS variable. <a href="#">[more]</a> .                     |
| <code>perpetual</code> :    | If true allows the behavior to to run even after it has completed. <a href="#">[more]</a> .                       |
| <code>post_mapping</code> : | Re-direct behavior output normally to one MOOS variable to another instead. <a href="#">[more]</a> .              |
| <code>priority</code> :     | The priority weight of the behavior. <a href="#">[more]</a> .                                                     |
| <code>pwt</code> :          | Same as <code>priority</code> .                                                                                   |
| <code>runflag</code> :      | A MOOS variable and a value posted when a behavior has met its conditions. <a href="#">[more]</a> .               |
| <code>spawnflag</code> :    | A MOOS variable and a value posted when a behavior is spawned. <a href="#">[more]</a> .                           |
| <code>spawnxflag</code> :   | A MOOS variable and a value posted when a behavior is spawned. <a href="#">[more]</a> .                           |
| <code>templating</code> :   | Turns a behavior into a template for spawning behaviors dynamically. <a href="#">[more]</a> .                     |
| <code>updates</code> :      | A MOOS variable from which behavior parameter updates are read dynamically. <a href="#">[more]</a> .              |

*Listing 1.2: Configuration Parameters for the MemoryTurnLimit Behavior.*

| Parameter                  | Description                                                                                                                                                                                                                                                                               |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>memory_time</code> : | The duration of time for which the heading history is maintained and heading average calculated. The default value is -1, indicating that the parameter is un-set. In this case the behavior will not produce an objective function.                                                      |
| <code>turn_range</code> :  | The range of heading valuesdeviating from the current heading average outside of which the behavior reflects sharp penalty in its objective function. The default value is -1, indicating that the parameter is un-set. In this case the behavior will not produce an objective function. |

*Listing 1.3: Example Configuration Block.*

```

Behavior = BHV_MemoryTurnLimit
{
  // General Behavior Parameters
  // -----
  name          = mem_turn_limit      // default
  pwt           = 100                  // default
  condition     = MODE=TRANSITING     // example
  condition     = ARRAY=connected     // example
  updates       = MEM_TURN_UPDATES    // example

  // Parameters specific to this behavior
  // -----
  memory_time = 60                    // example (seconds)
  turn_range  = 30                    // example (degrees)
}

```

## 1.2 Calculation of the Heading History

The heading history is maintained locally in the behavior by storing the currently observed heading and keeping a queue of  $n$  recent headings within the `memory_time` threshold. The heading average calculation below handles the issue of angle wrap in a set of  $n$  headings  $h_0 \dots h_{n-1}$  where each heading is in the range  $[0, 359]$ .

$$\text{heading\_avg} = \text{atan2}(s, c) \cdot 180/\pi,$$

where  $s$  and  $c$  are given by:

$$s = \sum_{k=0}^{n-1} \sin(h_k \pi / 180), \quad c = \sum_{k=0}^{n-1} \cos(h_k \pi / 180).$$

The vehicle turn radius  $r$  is not explicitly a parameter of the behavior, but is given by:

$$r = v / ((u/180)\pi),$$

where  $v$  is the vehicle speed and  $u$  is the turn rate given by:

$$u = \text{turn\_range} / \text{memory\_time}.$$

The same turn radius is possible with different pairs of values for `turn_range` and `memory_time`. However, larger values of `turn_range` allow sharper initial turns but temper the turn rate after the initial sharper turn has been achieved.

## 1.3 Variables Published

The only variable published by this behavior is `MEM_TURN_AVG` which indicates the present heading average, rounded to the nearest integer.

## 1.4 The MemoryTurnLimit Objective Function

A Rendering of the MemoryTurnLimit Objective Function

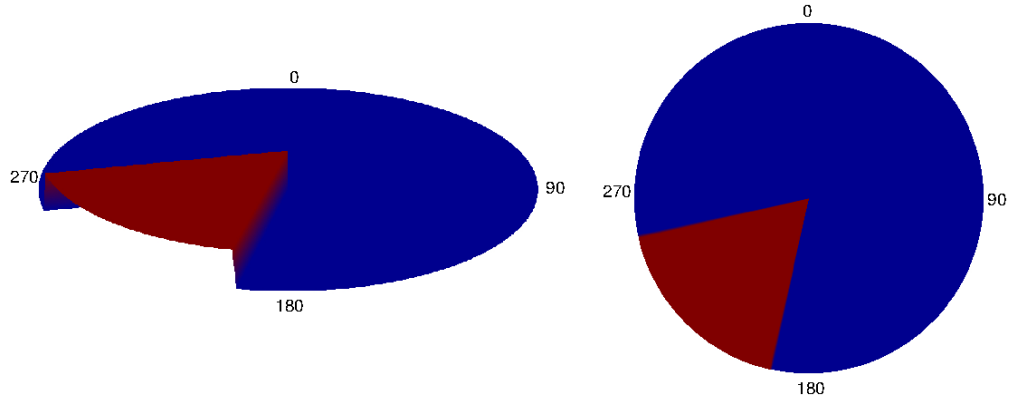


Figure 1: **The MemoryTurnLimit objective function:** The objective function produced by the MemoryTurnLimit behavior is defined over possible heading values. Depicted here is an objective function formed when the recent heading history is 225 degrees and the **turn\_range** parameter is set to 30 degrees. The resulting objective function highly favors headings in the range of 190-240 degrees. On the right is a "birds-eye" view of the function, and on the left the function is viewed at an angle to appreciate the 3D quality of the function. Higher (red) values correspond to higher utility.